

Transactions & ACID

A **transaction** is one unit of work as the database sees it: a sequence of **reads** and **writes** ending in **commit** or **abort**. Concurrency buys throughput and lets transactions corrupt each other; ACID is what the DBMS promises back.

Atomicity: all actions happen or none do, no half-sold ticket after a crash. **Consistency**: valid state to valid state. **Isolation**: your transaction runs *as if* it were alone; this is the hard one. **Durability**: once it commits it survives a power cut.

Notation. R1(A) T1 reads A W2(B) T2 writes B C1 / A1 T1 commits / aborts S1(A) / X1(A) T1 takes a shared / exclusive lock on A U1(A) T1 releases its lock on A.

Schedules

A **schedule** of $T_1 \dots T_n$ orders all their operations, with one rule: *each transaction's own operations keep their original relative order*. Shuffle the decks together; never shuffle a deck against itself.

Serial: no interleaving; correct by definition, but the CPU idles during I/O. **Equivalent**: for *every* starting state, both schedules leave the same final state. **Serializable**: equivalent to *some* serial order, it need not *be* serial, only indistinguishable from serial.

Conflicts: the one rule behind all three anomalies

Two operations **conflict** when all three hold: (1) different transactions, (2) same object, (3) at least one is a **write**. Hence R1(A)/W1(A), W1(A)/W2(B) and R1(A)/R2(A) never conflict.

Pattern	Name	What breaks
WR	Dirty read	T2 reads what T1 wrote but never committed; T1 aborts, so T2's value never existed.
RW	Unrepeatable read	T1 reads A, T2 changes it, T1 re-reads: a different answer mid-transaction. Kills read→decide→write.
WW	Lost update	T2 overwrites A mid-flight, T1 then writes B: one update vanishes, invariants break.

Conflict Serializability & the Precedence Graph

“Same effect for every database state” cannot be checked directly, so we use a stricter, mechanical proxy.

Conflict equivalent: same actions, and *every* pair of conflicting actions appears in the same order.

Conflict serializable: conflict equivalent to some serial schedule, i.e. you can reach a serial order by swapping only *adjacent non-conflicting* actions.

Do this every time:

1. One node per transaction.
2. Scan the schedule left to right. For every conflicting pair, draw $T_i \rightarrow T_j$ where T_i 's operation comes *first*. Label the edge with the object.
3. Look for a cycle. **No cycle** $\Rightarrow$ conflict serializable, and any topological order *is* an equivalent serial schedule. **Cycle** $\Rightarrow$ not conflict serializable.

Aborts: three fences, each stricter than the last

Add one abort and a “correct” schedule can become impossible to undo. If T_j read what T_i wrote, aborting T_i forces aborting T_j , a **cascading abort**; and if T_j already committed, nothing can be done.

Recoverable	Commit only after everyone you read from has committed. You never un-commit; cascades still possible.
ACA	Read only <i>committed</i> data. One abort stays one abort. ACA $\Rightarrow$ recoverable, not the reverse.
Strict	T_i 's value is neither read <i>nor overwritten</i> until T_i ends. Undo = restore the before-image.

Serial $\subset$ Strict $\subset$ ACA $\subset$ Recoverable; conflict serializability is a *separate axis* crossing all of them. The **log** (every write recorded before it is applied) is what makes undo possible at all.

Locking & Two-Phase Locking

A **locking protocol** makes non-serializable schedules *unreachable*: every object has a lock; acquire before you touch; wait if an incompatible lock is held; release when done. *When you release* is the entire design question.

held $\rightarrow$ / want $\downarrow$	None	S	X
S	OK	OK	conflict
X	OK	conflict	conflict

S = shared (read), many holders. X = exclusive (write), one holder. This is the conflict rule in a uniform.

Locks alone are not enough. If T1 releases A and *later* grabs B, T2 can slip past it on A and get ahead of it on B — a cycle, with every lock correctly taken.

2PL: once a transaction releases *any* lock it may never acquire another — growing phase, lock point, shrinking phase. Sorting by lock point gives the serial order, so no cycle can form.

Strict 2PL: release *all* locks at once, at commit/abort. No shrinking phase — a cliff.

Strict 2PL buys all four at once — conflict serializable, recoverable, avoids cascading aborts, strict — which is why it, not plain 2PL, is the default in essentially every commercial DBMS. It does *not* prevent **deadlock** (T1 holds A wants B, T2 holds B wants A); the DBMS detects that and aborts a victim.

Thrashing. Concurrency helps until blocking dominates; past that point throughput *drops* as load rises. If more than $\sim 30\%$ of transactions are blocked, run fewer concurrently. Three levers: **lock smaller things** (rows, not tables), **hold locks less time** (no network calls inside a transaction), **kill hot spots**.

Every box opens with one example worked out. Copy the method, not the answer. Notation: $R1(A)$ = T1 reads A, $C1$ = commit, $A1$ = abort.

Box 1 Name That Anomaly (7 min, pairs)

For each schedule: identify the conflicting pair that causes the trouble, write the pattern (**WR** / **RW** / **WW**), and say in one line what the user actually sees go wrong.

(a) $R1(A)$ $W1(A)$ $R2(A)$ $W2(A)$ $C2$ $A1$

(worked for you)

Pair: $W1(A)$ then $R2(A)$. Pattern: **WR**. Breaks: T2 read A before T1 committed, then T1 aborted — but T2 has already committed, so the DB holds work derived from a value that never existed.

(b) $R1(A)$ $R2(A)$ $W2(A)$ $C2$ $R1(A)$ $W1(A)$ $C1$

Pair: _____ Pattern: _____ Breaks: _____

(c) $W1(A)$ $W2(A)$ $W2(B)$ $C2$ $W1(B)$ $C1$ (invariant: A and B must always be equal)

Pair: _____ Pattern: _____ Breaks: _____

(d) **The ticket demo.** Both servers read `remaining` $\rightarrow 1$, checked > 0 , then wrote 0. Write it as a schedule on object R, and name the pattern.

Schedule: _____ Pattern: _____

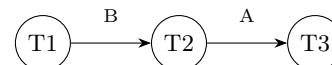
Box 2 Building a Precedence Graph (12 min, discuss)

(a) $S1 = R2(A)$ $R1(B)$ $W2(A)$ $R3(A)$ $W1(B)$ $W3(A)$ $R2(B)$ $W2(B)$

(worked for you)

Method: walk left to right and, for each object separately, list every cross-transaction pair in which at least one side writes. Two rows are done — finish the rest before you look at the graph.

Obj	Pair, in schedule order	Type	Edge
A	$W2(A)$ then $R3(A)$	WR	$T2 \rightarrow T3$
A	$R2(A)$ then $W3(A)$	RW	$T2 \rightarrow T3$
A	$W2(A)$ then $W3(A)$	_____	_____
B	$R1(B)$ then $W2(B)$	_____	_____
B	$W1(B)$ then $R2(B)$	_____	_____
B	$W1(B)$ then $W2(B)$	_____	_____



Acyclic $\Rightarrow$ conflict serializable. Equivalent serial order, read straight off the arrows: **T1, T2, T3**.

(b) **YOUR TURN** $S2 = W3(A)$ $R1(A)$ $R1(B)$ $W2(B)$ $W2(A)$. Fill the table, draw the edges, decide.

Obj	Pair, in schedule order	Type	Edge
A	$W3(A)$ then $R1(A)$	_____	_____
A	_____	_____	_____
A	_____	_____	_____
B	_____	_____	_____

Careful: three of the four rows involve object A. Do not stop at the first pair you find.



Cycle? ☐ yes ☐ no
Serial order: _____

Box 3 A Cycle, and How to Break It (8 min)

(a) $S3 = R1(A) R2(B) W1(B) W2(A)$. This schedule is *not* conflict serializable.

Edge from object A: _____ Edge from object B: _____

Why can no serial order exist? _____

(b) Swap or move exactly *one* operation of $S3$ so that the schedule becomes conflict serializable. Write the new schedule, its two edges, and its serial order.

(c) $S3$ is a real anomaly, not just a bad drawing. Which of the three patterns from Box 1 does each edge correspond to?

A-edge: _____ B-edge: _____

Box 4 Recoverable, ACA, Strict (10 min)

Q1. Does any transaction *commit* before a transaction whose uncommitted write it read? $\Rightarrow$ if yes, **not recoverable**.

Q2. Does any transaction *read* a value written by a still-uncommitted transaction? $\Rightarrow$ if yes, **not ACA**.

Q3. Does any transaction read *or overwrite* a value written by a still-uncommitted transaction? $\Rightarrow$ if yes, **not strict**.

Schedule	Recov.	ACA	Strict	Which question failed, and why
$W1(A) C1 R2(A) W2(A) C2$ (<i>worked for you</i>)	yes	yes	yes	none — nothing is touched before its writer commits
$W1(A) R2(A) W2(A) C1 C2$	_____	_____	_____	
$W1(A) R2(A) W2(A) C2 C1$	_____	_____	_____	
$W1(A) W2(A) C1 C2$	_____	_____	_____	

(b) Row 3 is the dangerous one. Suppose $T1$ aborts instead of committing. What *should* happen to $T2$, and why is the DBMS powerless to do it?

.....

(c) Row 4 has no dirty *read* at all — yet it still fails one of the three. Which one, and what makes undo hard here?

.....

Box 5 2PL Fixing Lock Failure (14 min, discuss)

Both transactions lock every object before touching it and unlock as soon as they are done with it. Every lock is correct — the schedule is still non-serializable.

T1	T2
X(A), R(A), W(A), U(A)	X(A), R(A), W(A), U(A)
X(B), R(B), W(B), U(B), Commit	X(B), R(B), W(B), U(B), Commit

(a) Give the two edges and confirm the cycle.

On A: _____ On B: _____ Cycle? ☐ yes ☐ no

(b) Circle T1's *first unlock*. Does T1 acquire a lock after it? Which 2PL rule breaks, and what did that let T2 do?

.....

(c) **YOUR TURN** Rewrite it under **Strict 2PL** with the same operations, only lock placement changes. Mark where T2 is **denied** and waits.

T1	T2
X(A), R(A), W(A)	
_____	_____

(d) Tick every property the Strict 2PL version now has: ☐ conflict serializable ☐ recoverable ☐ avoids cascading aborts
☐ strict

The single rule that bought all four: _____